「模型即软件」之后，工程师如何从 Coding 技能栈多走一步

司潘俊楠

蚂蚁集团，基础智能技术部（百灵大模型），Model as Product 团队

18 届计科本硕校友。常用 id: cafe3310

接下来会聊什么

1. 百灵模型的介绍和故事

在做模型期间，我们学到了什么？

2. 被 LLM 重新定义的「软件行业价值」

看看 Anthropic 是怎么发篇 Blog 就击坠几个公司股价的，以及为什么。

3. 被 LLM 逐渐改变的「软件形态」

模型在开发领域为何提升如此迅速？这几年出现了什么新的软件架构？

4. 下一代「软件开发者」的实用技能：模型微调和自定义

什么变得廉价了、什么仍然紧缺？为什么需要模型自定义？

5. 也许有用的开源任务

我们给活动带来了什么可参与的任务。

1. 百灵模型的介绍和故事

在做模型期间，我们学到了什么？

2. 被 LLM 重新定义的「软件行业价值」

看看 Anthropic 是怎么发篇 Blog 就击坠几个公司股价的，以及为什么。

3. 被 LLM 逐渐改变的「软件形态」

模型在开发领域为何提升如此迅速？这几年出现了什么新的软件架构？

4. 下一代「软件开发者」的实用技能：模型微调和自定义

什么变得廉价了、什么仍然紧缺？为什么需要模型自定义？

5. 也许有用的开源任务

我们给活动带来了什么可参与的任务。

我们经常被问到的问题

> 「为什么要自己从头训练模型？」

数据、场景与商业考量

手握非开放的行业数据、面对不可外包的隐私场景、加上真实商业对智效比的要求 —— 决定了不能只调别人的模型

01 数据

有好的非开放数据，还需要转起 Data Scaling 飞轮

特定场景有好的非开放数据是优势，但「模型进入专业场景」，再「由真实交互持续产出新数据」，**这个 Data Scaling 的 Loop 只有自己训模型才能转起来**。

02 专业场景

严肃场景无法依赖外部模型服务

Coding 不是唯一 LLM 加成的场景。金融、医疗等严肃领域出于**隐私与合规**无法依赖外部模型；行业特定 Task（如医生追问病史、金融专业检索）**需要自己做训练 Task**。

03 商业逻辑

技术上要「能训个大的」，但做商业必须讲智效比

模型做好了，公司毕竟还是要做商业。在业务**成本、时延、准确率**的要求下，一方面要通过训大模型探明能力上限，另一方面必须**压缩 size 与激活量（部署成本，并发能力，推理成本）**，否则会无法支撑商业基本逻辑。

这些还只是业务层面的原因

自己做才能了解 Why 和 How

在模型生产中才能了解的经验

训练期习得的 Know-how 和 Know-why

语料配比	中英文语料训练配比如何权衡?
合成数据	合成数据的有效边界在哪里?
强化学习	如何控制模型在 RL 过程中的 Reward-Hacking?
模型架构	线性注意力能省多少钱?
评测	如何设计最优的评测 Rubrics? 人评还是 Model Judge?

→

技术，商业和产品经验

01 如何控制模型推理成本?

决定「做什么产品是划算的」,「做什么无法划算」

02 如何避免做出「SOTA 模型也无法满足」的产品?

避免尝试做出「原理上就无法达到实用性」的产品（许愿盒子式产品）

03 如何避免被评测或榜单误导效果?

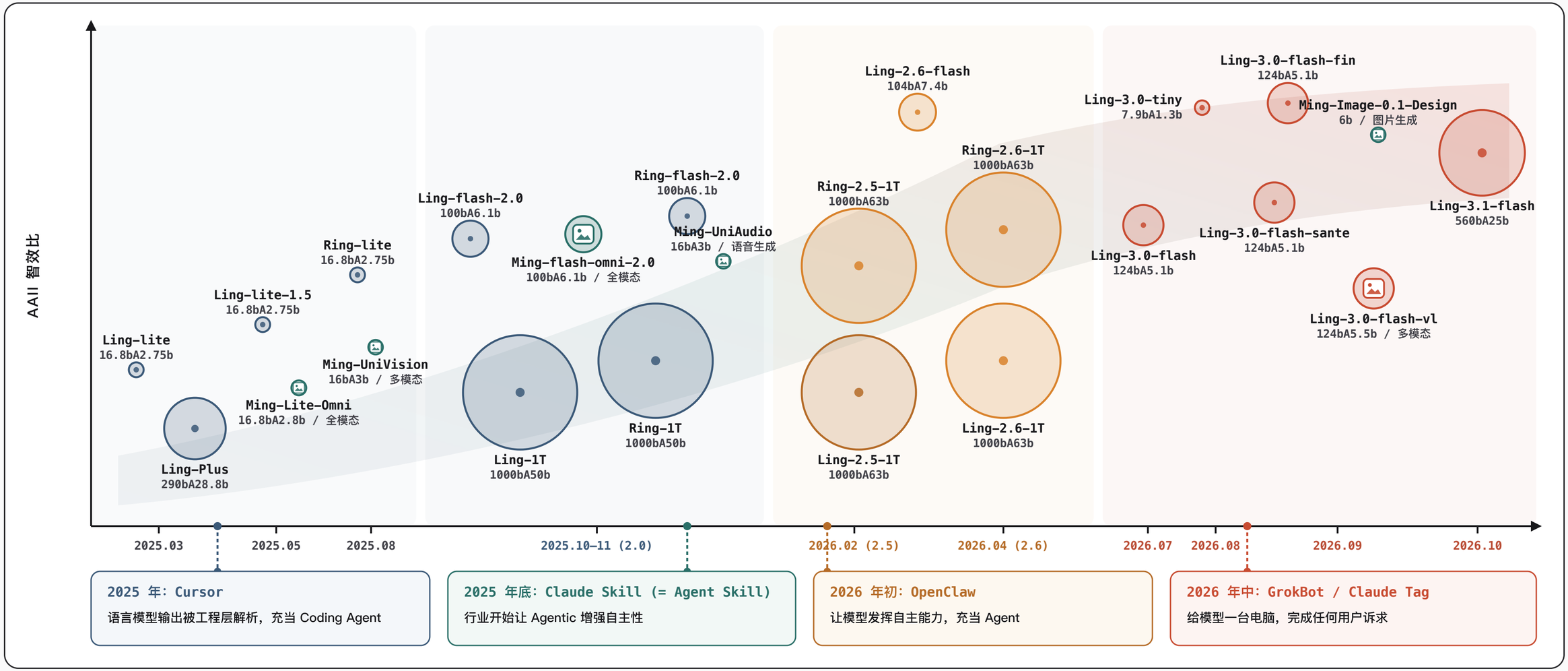
设计过评测才能看穿刷榜的水分。

Ling 系列模型演进(2025 – 2026)

从文本生成到 Agentic 执行。致力于推高智效比水平

文本生成 Agentic 多模态模型

参数标注说明：**124bA5.1b** = 总参数量 124B，单次激活参数量（Active）5.1B



我们学到了什么：大模型训练是一场超出算法的「大规模协同」

训练工程师的技能早已超出算法范畴，更像是综合大考

大家常以为的「训模型」

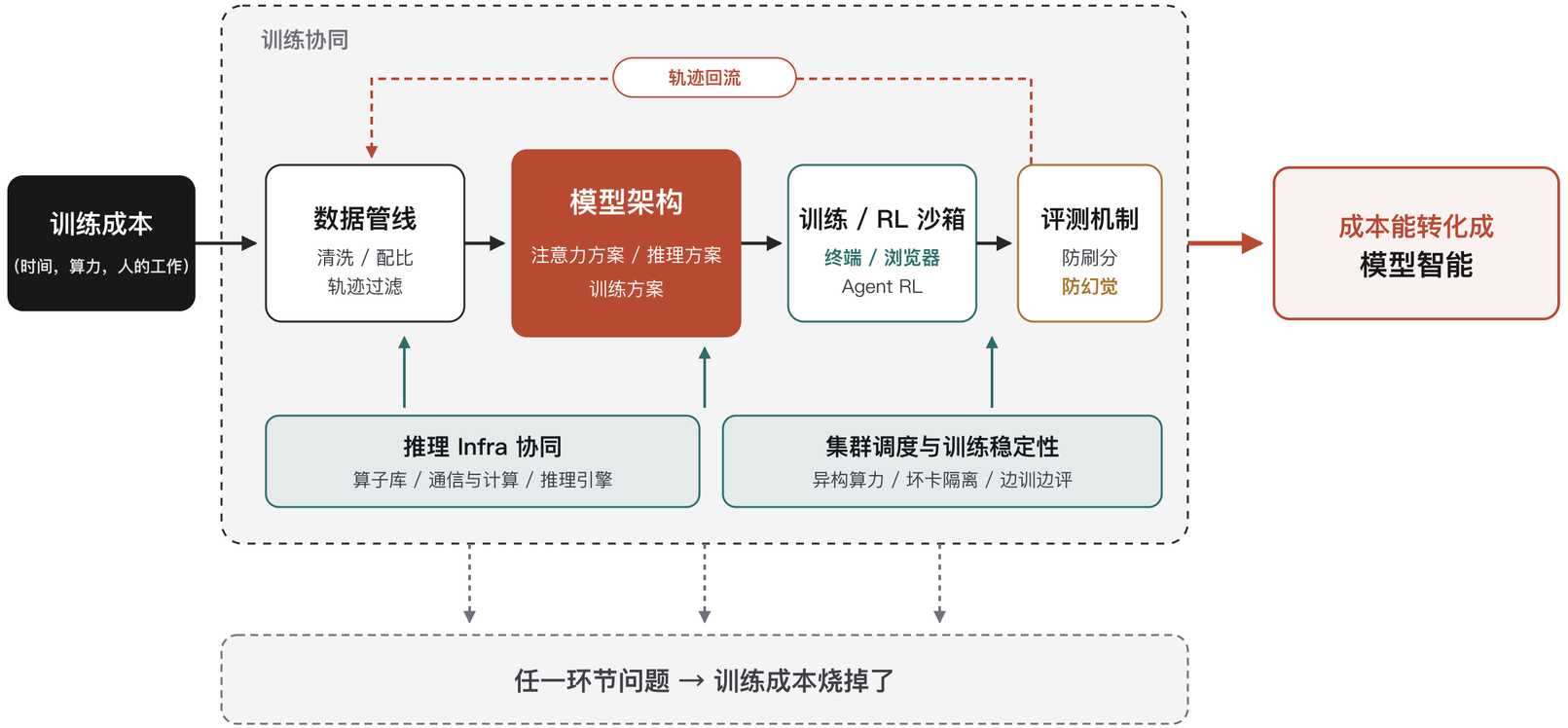
改网络结构、调超参、盯着 Loss 曲线下降

↓

真实模型生产中的「训模型」

以算法为主，但推理、调度、数据、环境、评测，协同解决问题。

任何一个环节成为短板，都会导致每次训练「烧掉」的钱被浪费掉。



我们学到了什么：不同阶段选择不同的策略

01

2025.10 – 2026.04（2.0 – 2.5 阶段）

学习 Scaling 阶段：先做个够大的

目标：验证工程 Co-design 能力，解决协同问题

Ling / Ring 1T

1000b，第一个 1T 开源模型



提升组织与协作效率：做大尺寸模型困难，稳定训练考验团队的工程协同效率与技术认知。

知道如何压缩：先看清能力上限才能做小型化。积累认知和方法，更好 teach 小模型。

02

2026.07 – 2026.10（3.0 – 3.1 阶段）

提升 Agentic 阶段：先做轻量的

目标：适应快节奏，快速验证新架构与后训流程后再 Scale up

3.0-flash → 3.1-flash

124bA5.1b → 560bA25b（从小到大）



行业变快了：Agentic 模型周期从去年的三四个月缩短到 **1 个月**；等半年训完一个超大模型再试新方案，节奏等不及。

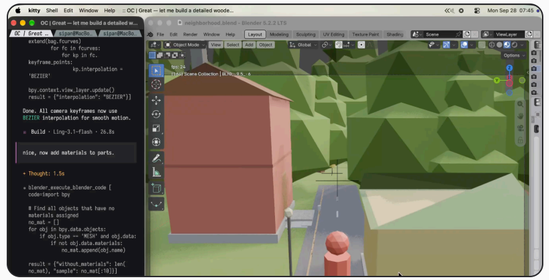
验证数据飞轮与 Post-train，做个小而好用的再 Scale Up：在 3.0-flash 上跑通 Agentic 后训练，做出小而好用的主力模型后，再向 3.1-flash（560bA25b）放大。

这个系列的现状，以及到哪里体验与获取

01 Agentic 语言模型

Ling-3.1-flash(-VL)

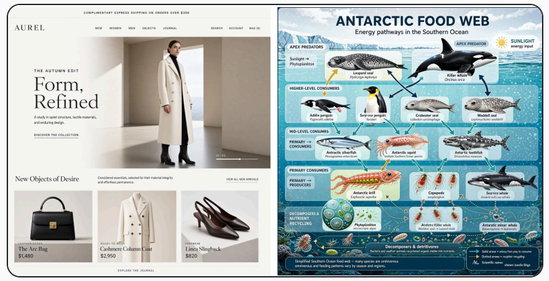
支持混合思考与工具调用，用于代码、视觉理解与复杂任务。



02 设计与图片生成模型

Ming-Image-0.1-Design

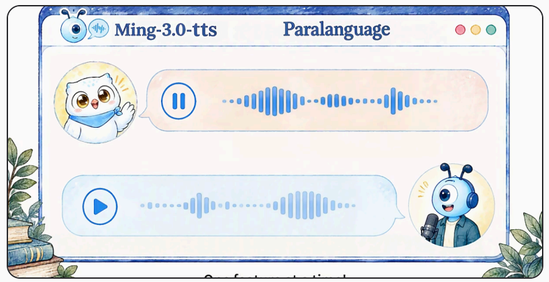
面向平面设计场景，支持中英文排版、海报生成与图层拆分。



03 语音模型

Ming-3.0-tts / Ming-3.0-asr

语音识别 / 语音生成 / 音频理解





官网	www.ant-ling.com
HuggingFace	huggingface.co/inclusionAI
GitHub	github.com/inclusionAI

或直接搜索「Ling LLM」

1. 百灵模型的介绍和故事

在做模型期间，我们学到了什么？

2. 被 LLM 重新定义的「软件行业价值」

看看 Anthropic 是怎么发篇 Blog 就击坠几个公司股价的，以及为什么。

3. 被 LLM 逐渐改变的「软件形态」

模型在开发领域为何提升如此迅速？这几年出现了什么新的软件架构？

4. 下一代「软件开发者」的实用技能：模型微调和自定义

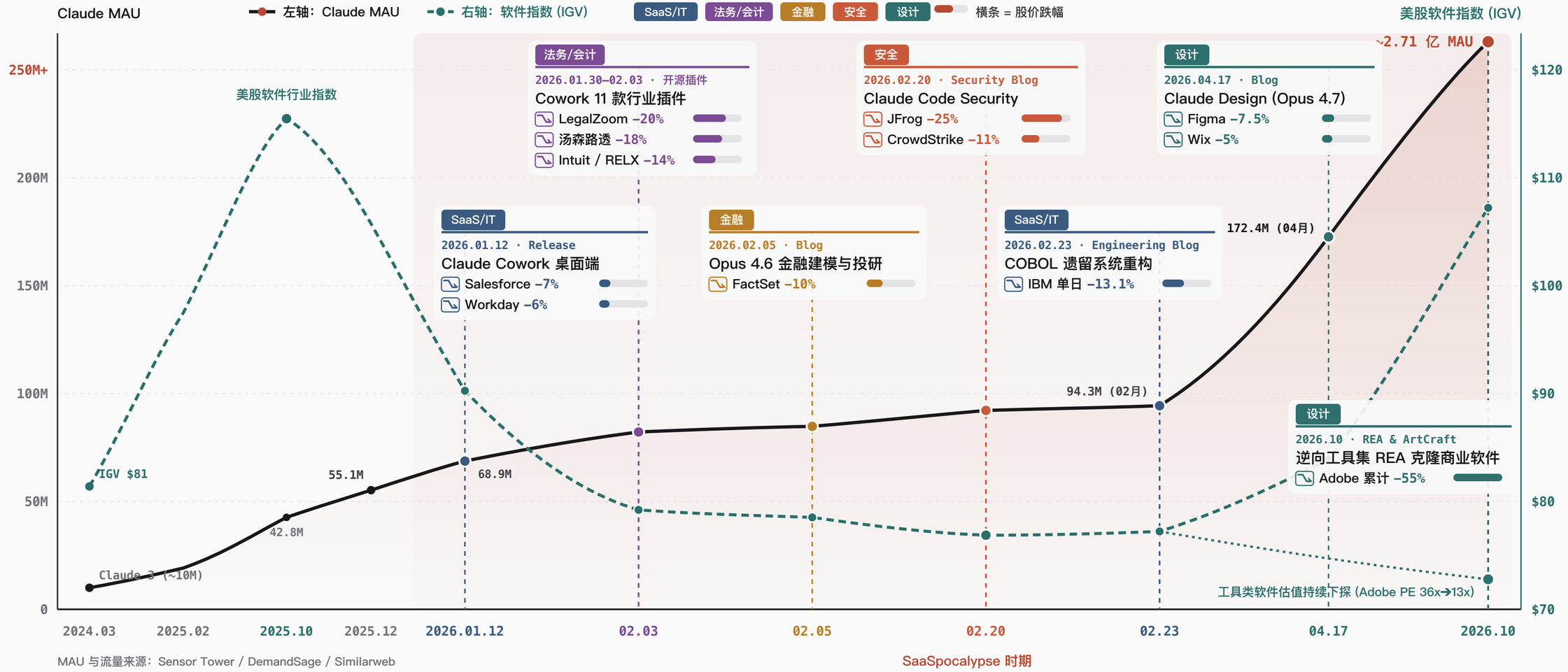
什么变得廉价了、什么仍然紧缺？为什么需要模型自定义？

5. 也许有用的开源任务

我们给活动带来了什么可参与的任务。

Anthropic：美股软件板块的「股价击坠机」

Claude 用户量快速上升 vs 美股软件指数 (IGV) 在 2026 年初大幅下跌，而后分化



这些软件公司也许当期利润仍在增长，但市场在重新评估「靠席位收费、堆复杂功能」的软件护城河还能维持多久。

软件价值的分化：什么变得不重要了，什么变得更重要了？

靠封装界面、按席位或堆工时收费的软件护城河变浅；承载组织协同、Agent 沙箱与底层系统软件反而更紧缺

什么变得不重要了（价值下降）

01 纯工具与桌面软件：复刻成本下降

Adobe −55% · Figma −7.5%

靠常年积累的复杂度做壁垒；现在只要输入输出明确，Coding Agent 配合逆向工具集有可能低成本复刻。

02 按席位收费的 SaaS：席位缩减

LegalZoom −20% · 汤森路透 −18%

传统 SaaS 增长建立在「客户多招人 → 多买席位」之上。人 + Agent 工作后「企业采购席位数」会自然减少。

03 被模型能力覆盖的专业技能软件

JFrog −25% · IBM −13.1%

LLM 直接扫描安全风险和漏洞，会冲击靠特征库收费的传统扫描器；LLM 理 COBOL 依赖、完成重构，会直接动摇按「人月工时」计费的技术咨询服务。

瓶颈转移到「组织协同」与「运行设施」

什么变得更重要了（价值上升）

01 组织上下文与协同基座

Atlassian · Palantir · GitHub

写代码不再是瓶颈、但需求确认、问题流转、权限审批与责任定位成了新瓶颈。

02 Agent Infra（如沙箱）：高并发执行需要隔离环境

Cloudflare · Vercel

轻量虚拟机沙箱（如 `Cloudflare Dynamic Workers`、`Vercel Sandbox`）、工具环境与运行时边界保护的 Infra 成为 Agent 工作的刚需。

03 基础与系统软件：Agent 依赖的底座，难以被轻易替换

Linux / K8s · 数据库 · Datadog

操作系统、数据库、网络能力与可观测系统，是 Agent 赖以运行的基础。其知识已经被 bake into 模型能力，无法轻易替换。

模型可以低成本重写「上层应用代码与操作界面」，但必须依赖「组织协同状态、Agent 沙箱与底层系统软件」才能运转。

1. 百灵模型的介绍和故事

在做模型期间，我们学到了什么？

2. 被 LLM 重新定义的「软件行业价值」

看看 Anthropic 是怎么发篇 Blog 就击坠几个公司股价的，以及为什么。

3. 被 LLM 逐渐改变的「软件形态」

模型在开发领域为何提升如此迅速？这几年出现了什么新的软件架构？

4. 下一代「软件开发者」的实用技能：模型微调和自定义

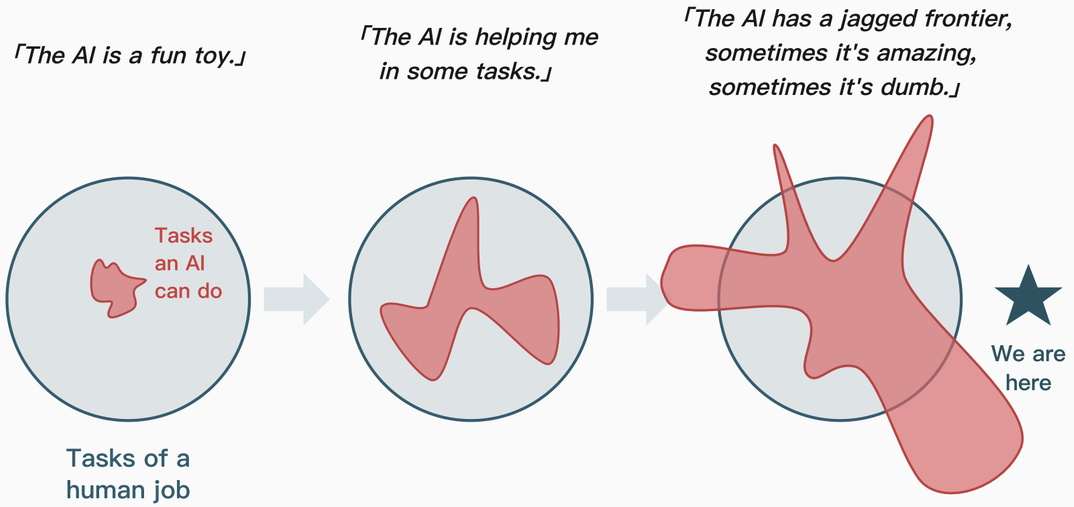
什么变得廉价了、什么仍然紧缺？为什么需要模型自定义？

5. 也许有用的开源任务

我们给活动带来了什么可参与的任务。

模型的「参差边界」：为什么各家都从 Coding 入手?

可验证性、通用杠杆与客观评判标准让 Coding 成为突破口



模型能力并不是均匀扩张的，而是长成参差不齐的形状。为什么各家都优先从软件开发入手?

01 任务可验证：编译器与测试能直接给反馈

「写文章」或「做方案」很难自动 Judge；但代码能不能过编译和单测，沙箱跑一下就能给出打分，非常适合做大规模强化学习。

02 通用性：代码是操纵数字世界的通用工具

模型一旦学会写代码和操作终端，就顺带学会了做数据分析、调外部接口、管理系统环境，甚至给自己写新工具。这是从对话模型走向通用 Agent 最直接的路径。

03 评价标准客观：较少审美或主观优劣争议

不同于艺术设计或写作容易有主观偏好争议，工程代码只要逻辑正确、测试通过、性能达标，不管是人写的还是模型写的其实都可用。

为什么模型的 Agentic 与开发能力提升这么快?

2026 年发展起来的 In-Environment RL 和「订阅用户」形成的训练飞轮

01 后训练沙箱环境逐渐丰富

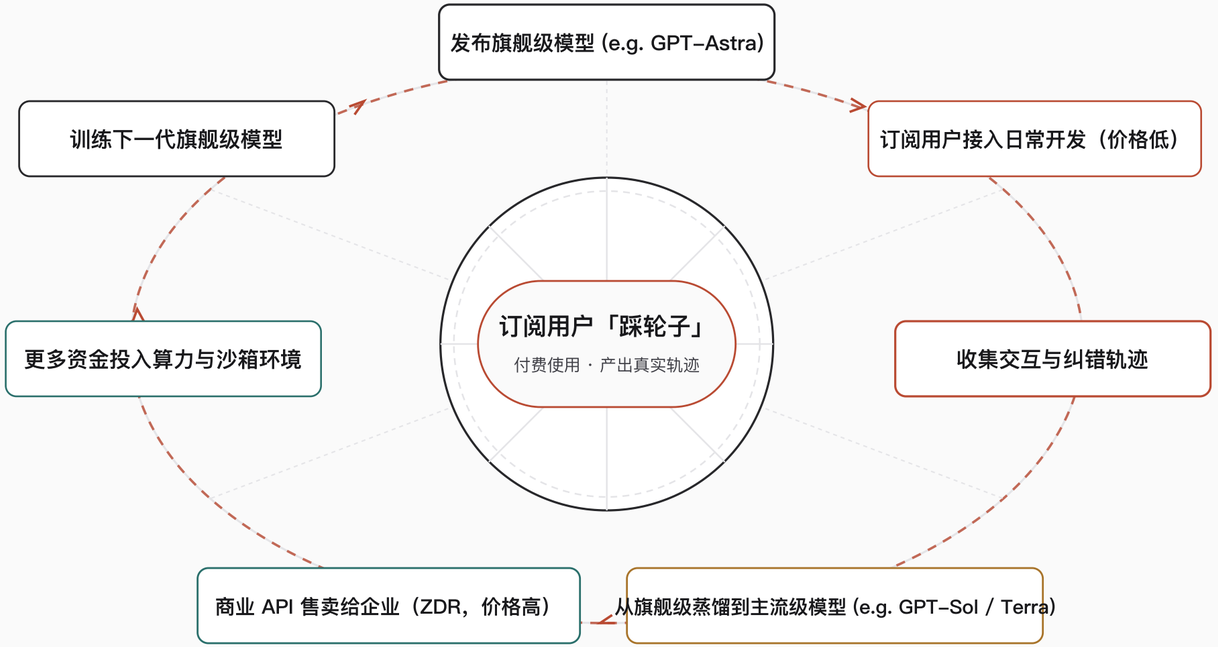
模型在真实沙箱（包含终端、浏览器等的测试环境）中大量练习「执行 → 报错 → 修改 → 调工具」，把原本需要手写的状态流转和异常处理训进了参数里。

02 后训练让模型输出更稳定，更容易接入工程系统

高强度强化学习收窄了输出分布，让模型在调工具和结构化输出上足够稳定，可以作为组件放进已有的系统里。

03 训练飞轮：付费开发者在日常使用中持续贡献训练数据

开发者付月费，在真实项目里写 Prompt、改错、跑测试；在非隐私模式下，这些高价值的交互轨迹被回收用来训练下一代模型。

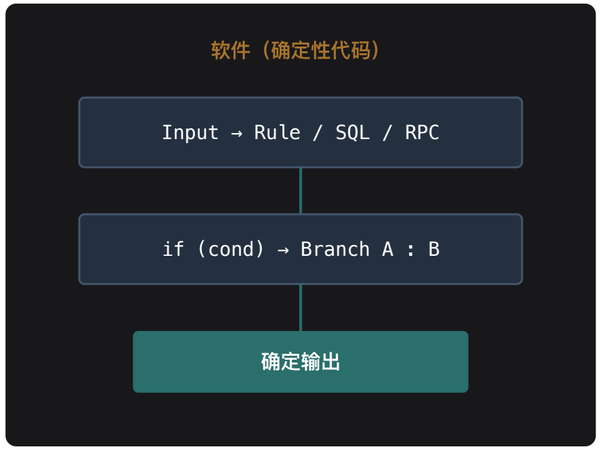


Coding 是模型自我迭代最快跑通的领域，可能也会是（所谓）RSI 的起点

软件架构的变化（一）：模型从外部服务变成主要功能

传统的「确定性软件」

ERP / 数据库 / 传统微服务

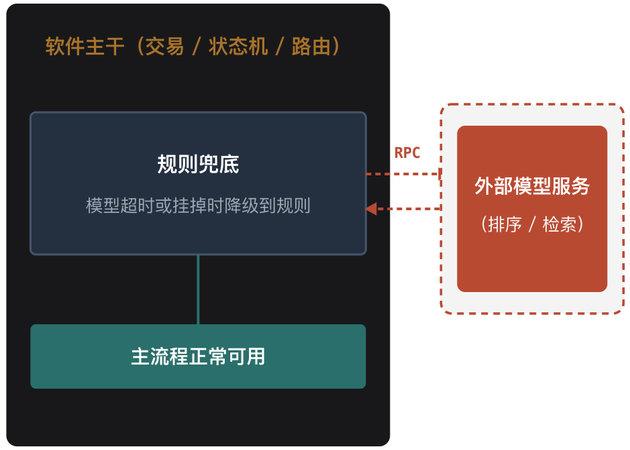


模型失效影响

给定输入得到确定输出

含模型服务（如搜推广等）的软件

模型作为软件外部的推荐 / 排序服务被调用



模型失效影响

模型挂掉可走规则降级，软件照常可用

LLM Chatbot / Task Agent

模型成为软件的主要功能



模型失效影响

若模型不工作，软件本身不可用

功能上：新出现的软件架构，LLM 逐渐变成软件主要功能

软件架构的变化（二）：流程控制权从代码转移给模型

代码编排工作流 (LangGraph / n8n 类)

代码定义好状态图，按固定步骤调用模型



流程控制权

控制权在代码：由代码决定每一步调哪个模型、走哪个分支

Agent Harness（模型掌控流程，代码防御问题）

模型自己决定执行步骤，代码只提供工具和环境



流程控制权

控制权在模型：模型自己决定「下一步做什么」，代码提供工具、维护环境

正在发生：Decision Model 接管部分「智能 if」

用轻量 Decision / Rank 模型替代代码里的分支判断



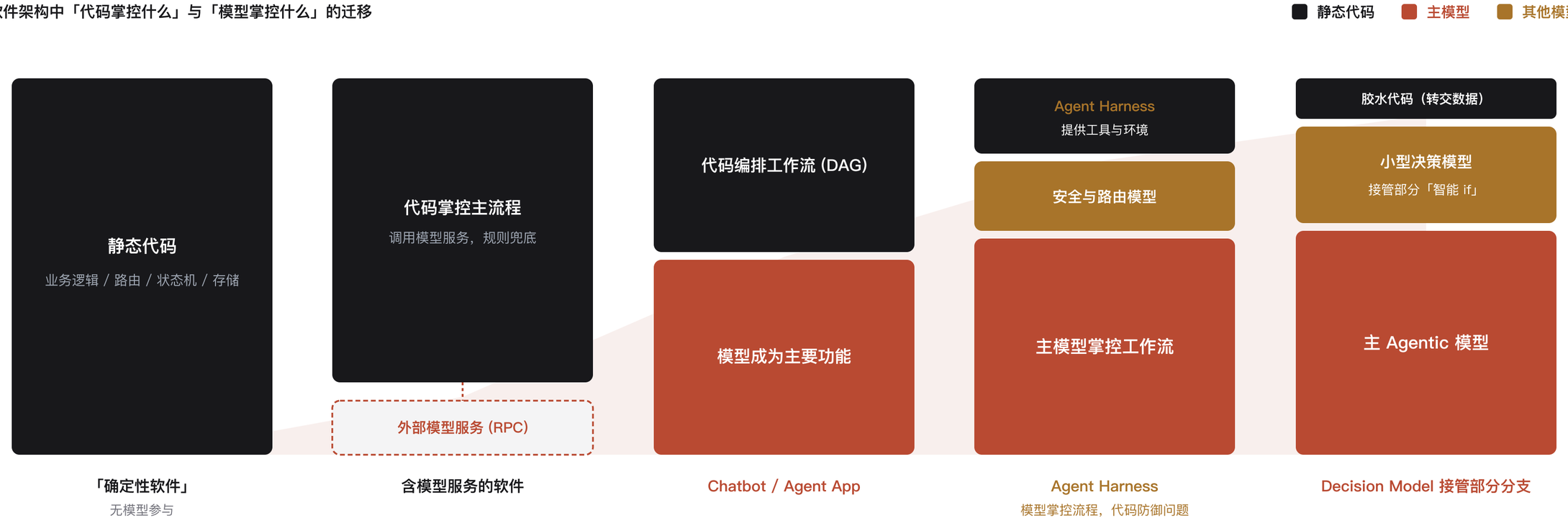
流程控制权

部分分支交给小模型，代码「转交数据」

控制权上：新出现的软件，LLM 掌握更多逻辑控制权

近年出现的以 LLM 为内核的软件架构

软件架构中「代码掌控什么」与「模型掌控什么」的迁移



所以有一些说法会称之为「软件代码被炼进模型里了」

1. 百灵模型的介绍和故事

在做模型期间，我们学到了什么？

2. 被 LLM 重新定义的「软件行业价值」

看看 Anthropic 是怎么发篇 Blog 就击坠几个公司股价的，以及为什么。

3. 被 LLM 逐渐改变的「软件形态」

模型在开发领域为何提升如此迅速？这几年出现了什么新的软件架构？

4. 下一代「软件开发者」的实用技能：模型微调和自定义

什么变得廉价了、什么仍然紧缺？为什么需要模型自定义？

5. 也许有用的开源任务

我们给活动带来了什么可参与的任务。

从刚才的情况看：什么已经「供大于求」，什么「仍然被需要」？

已经呈现「供大于求」的能力

01 **Code 的编写能力**
把明确需求翻译成函数与样板代码

02 **Review 具体代码的能力**
逐行检查拼写、格式或局部语法问题

03 **对 API 的记忆和背诵**
记忆标准库的签名、配置与写法

04 **「会调用模型」**
调接口、写基础 Prompt，开发简单的 Agent App

也许仍然被需要的能力

01 **Verify 的能力**
计算机基础知识，尤其是底层的。可以不会被模型幻觉欺骗，能 verify 模型的解决方案。

02 **审美和脱离平均值的能力（所谓的 Taste）**
自回归模型天然趋向「训练数据的平均值」；分辨好坏、跳出平庸的模板，依然要靠人。

03 **定义任务、定制模型、优化任务的能力**
定义要做的任务、定义金标准、「什么是好」，并在成本和耗时限制下定制模型以发挥最优效果。

为什么有了通用大模型还需要定制小模型？高频分支判断（Decision Model）、特定领域规范与低成本/低延迟约束，都离不开轻量模型定制。

1. 百灵模型的介绍和故事

在做模型期间，我们学到了什么？

2. 被 LLM 重新定义的「软件行业价值」

看看 Anthropic 是怎么发篇 Blog 就击坠几个公司股价的，以及为什么。

3. 被 LLM 逐渐改变的「软件形态」

模型在开发领域为何提升如此迅速？这几年出现了什么新的软件架构？

4. 下一代「软件开发者」的实用技能：模型微调和自定义

什么变得廉价了、什么仍然紧缺？为什么需要模型自定义？

5. 也许有用的开源任务

我们给活动带来了什么可参与的任务。

这就是为什么我们带来的任务是：「自定义一个小型模型」

这虽然和大模型训练在规模上尚有一些区别，但在任务本质上并没有差异

Q1 从自己的兴趣出发

「有什么任务，是自己想让一个模型做得更好的？」

Q2 教给模型新能力（或强化已有能力）

「有什么是模型本来难以做到、你希望教给模型的？」

具体要做的事情：

01

选题与定义任务

选定一个具体场景，定义好任务输入和期望输出。

→

02

构造数据与评测方案

定义「什么是好」，清洗/合成训练数据和验证方案。

→

03

微调训练与效果对比

评估设备限制，用 MLX / AReno 等完成微调，验证微调前后的效果提升。

任务入口

Ling-3.0-tiny 微调和自定义任务

将一个轻量级 LLM 自定义成自己所需的。

如果对定制 Ling 不感兴趣，也可以试试看定制类似 Qwen、Gemma 等模型（不过就无法提交到 Ling-Cookbook 了）。



Ling-Cookbook 轻训营课题

`https://github.com/inclusionAI/ling-cookbook/issues/11`



建议使用 AReno 进行后训练和微调

`https://github.com/inclusionAI/AReno`